

The Agentic Platform Reference Architecture

A Needs-Driven Approach

Author: Raphaël MANSUY / Adham SERSOUR / Formalisation atelier de travail semaine 50 / 2025

Published: December 2025

Executive Summary

Organizations deploying autonomous AI agents face eight fundamental challenges. The Agentic Platform is the infrastructure layer that addresses them systematically.

Need	Core Question
Operational Reliability	How do agents run continuously, survive failures, and scale cost-efficiently?
Security & Trust	How do we prevent agents from taking unauthorized actions?
Governance & Compliance	How do agents operate within policies and regulations with full auditability?
Data Foundation & Factuality	How do agents access verified, high-quality data to make sound decisions?
Intelligence & Reasoning	How do agents reason effectively and apply appropriate strategies?
Integration & Connectivity	How do agents connect to tools, data, other agents, and humans?
Agent Exposition & Access	How do users and systems invoke and interact with agents?
Developer Productivity	How do teams build, test, deploy, and improve agents efficiently?

This document explores each need in depth, then maps them to a coherent architecture.

1. Context: Why These Needs Matter Now

Modern LLMs score 86%+ on MMLU, 85%+ on HumanEval, and 90%+ on GSM8K—benchmarks showing they can handle structured analytical tasks.

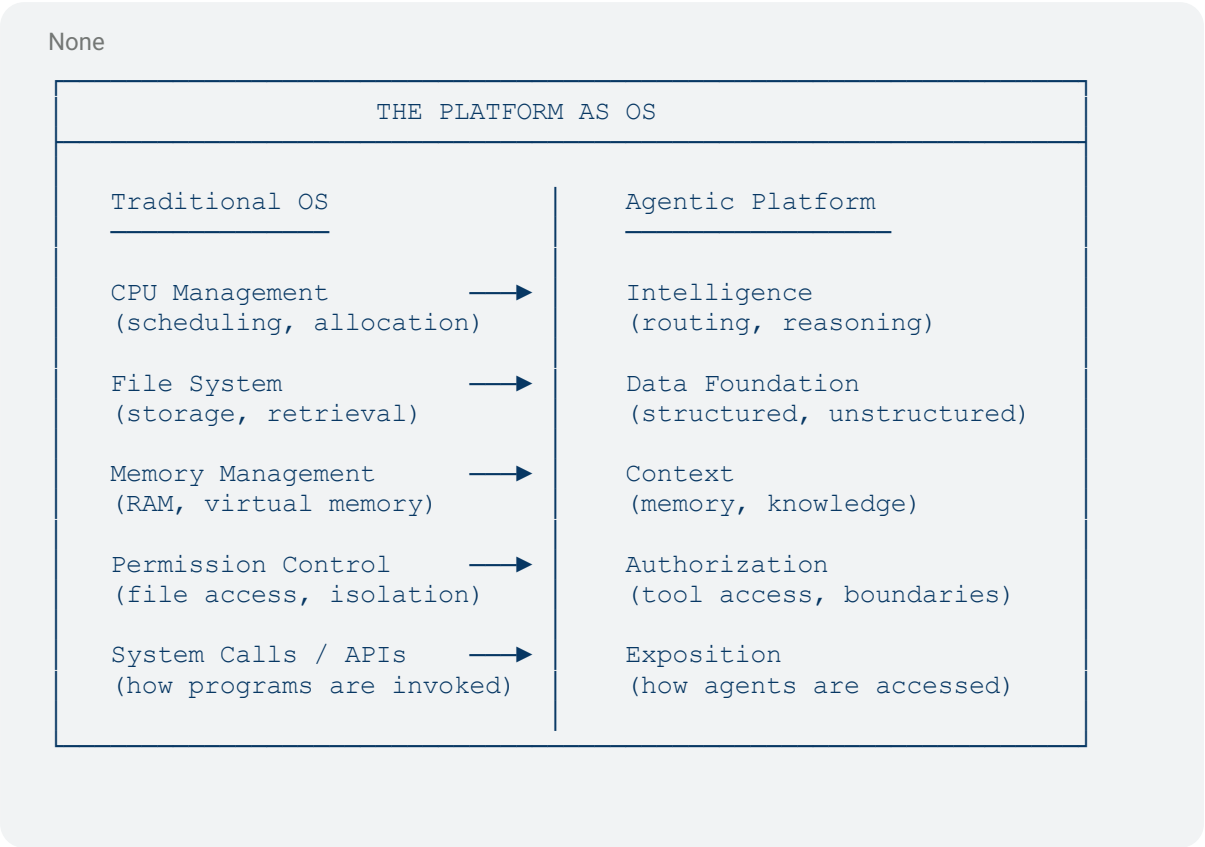
But capability ≠ production-readiness.

Raw LLMs are stateless, insecure, ungoverned, ungrounded, isolated, and invisible. The Agentic Platform bridges this gap.

The Operating System Analogy

A traditional operating system manages CPU cycles, memory allocation, and permissions for software processes.

*An Agentic Platform manages **Intelligence** (model routing, reasoning), **Data** (structured/unstructured, verification), **Context** (memory, knowledge), **Authorization** (permissions, boundaries), and **Exposition** (how agents surface to users and systems) for autonomous AI processes.*



2. Framework vs. Platform: A Necessary Clarification

Frameworks (LangChain, CrewAI, AutoGen, Google ADK) are code libraries for building agent logic. They answer: *How do I write code that makes an LLM behave as an agent?*

Platforms are infrastructure for running agent logic in production. They answer: ***How do I operate agents reliably, securely, at scale, and accessible to users?***

Frameworks run *on* platforms. A team might use LangChain to build agent logic, then deploy that logic to an Agentic Platform that provides durable execution, security, data access, observability, and exposition.

The eight needs that follow are platform-level concerns. Frameworks provide building blocks; platforms provide the production environment.

Need 1: Operational Reliability

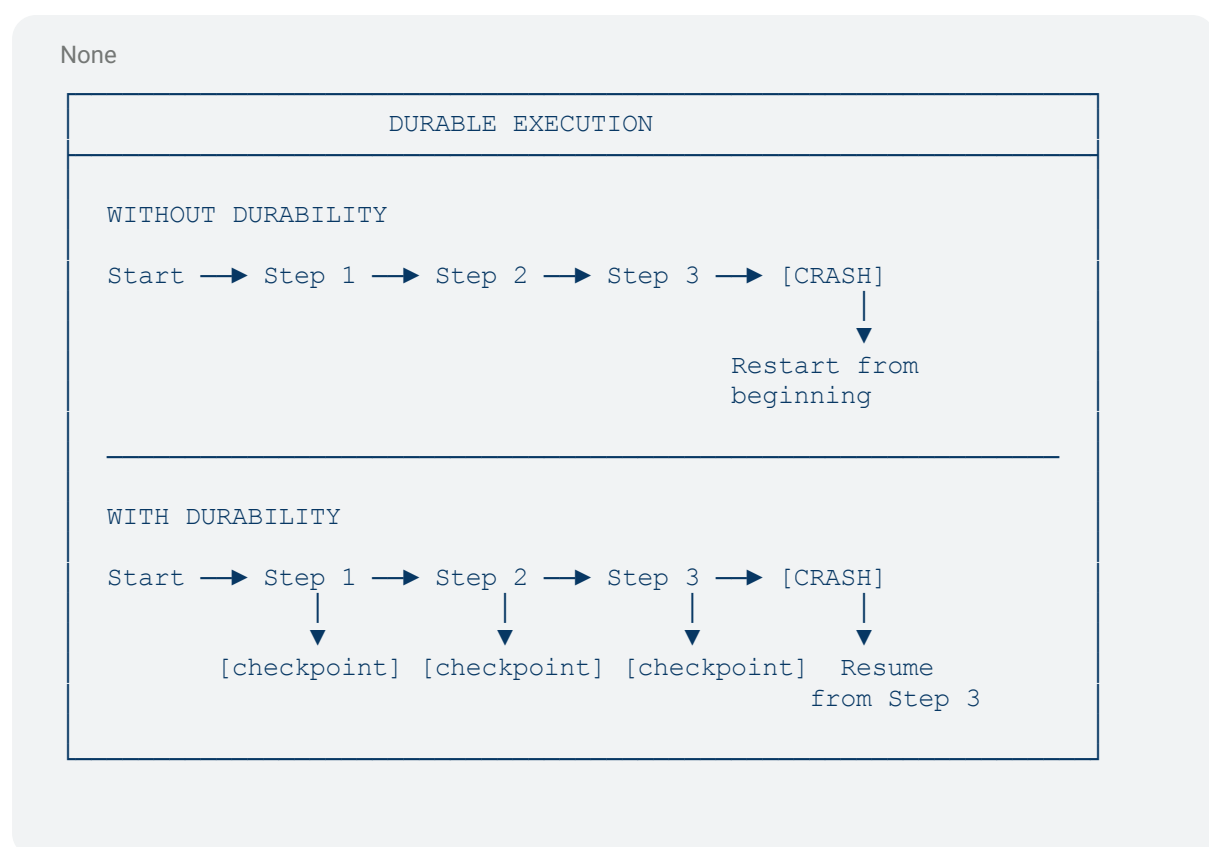
The Problem: Agents must run continuously, survive failures, scale with demand, and operate cost-efficiently around the clock.

1.1 Durability: Surviving Failures

Complex agent tasks may span hours or days: researching a market, analyzing contracts, onboarding a customer. If the underlying compute node fails mid-task, what happens?

Without durability, the agent restarts from scratch. Work is lost. Costs double.

Durable Execution checkpoints agent state at each significant step. If a node fails, another node resumes from the last checkpoint. The agent continues rather than restarts.



Implementation: Temporal, Azure Durable Functions, or custom checkpoint systems on reliable storage.

1.2 Scalability: Handling Variable Load

Agent workloads are bursty. A customer service agent might handle 10 requests per minute normally, then 1,000 during a product outage.

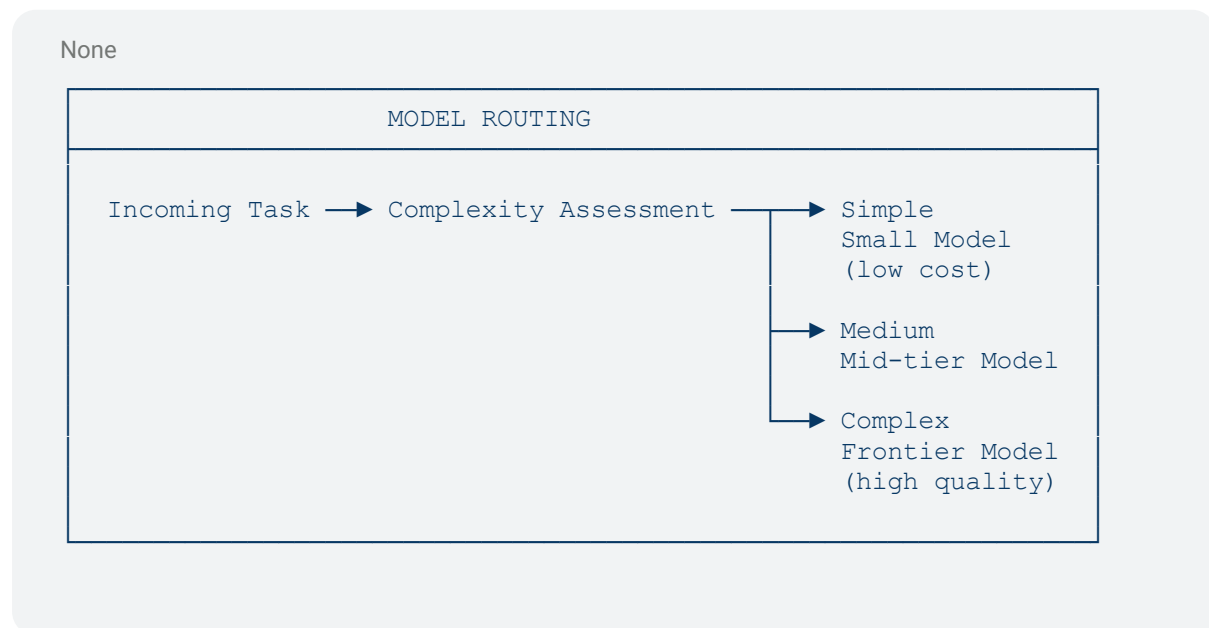
Elastic Scaling adjusts compute resources based on demand. The platform provisions additional agent instances when load increases and releases resources when load decreases.

Resource Quotas prevent runaway costs. An agent in an infinite loop should not consume unbounded resources. Quotas cap compute time, memory, token consumption, and API calls.

1.3 Cost Efficiency

Agent costs span multiple dimensions: compute, LLM tokens, storage, network, and human oversight.

Model Routing is the highest-impact cost lever. Not every task requires a frontier model. Simple classification might use a small, fast model; complex reasoning uses a larger model. Intelligent routing can reduce token costs by 50-80%.



Additional levers include caching (avoiding repeated identical requests), batching (grouping related requests), and right-sizing compute resources.

1.4 Observability

When an agent fails or produces unexpected results, operators need answers.

Distributed Tracing follows requests across agents, tools, and services. Metrics and Alerting track success rates, latencies, and costs in real-time. Reasoning Capture stores the chain-of-thought behind decisions—essential because agent behavior emerges from reasoning, not deterministic code.

Need 2: Security & Trust

The Problem: Agents must be prevented from taking unauthorized actions—even if instructed to do so through prompt manipulation.

2.1 The Architectural Security Imperative

Traditional application security assumes deterministic code execution. If you don't write code to delete accounts, the application won't delete accounts.

Agent security is fundamentally different. Behavior emerges from model weights and prompt inputs. A sufficiently clever prompt injection might convince an agent to attempt any action the underlying tools permit.

Security must be architectural, not prompt-based. Instructing an agent to "never delete accounts" in its system prompt is not security—it's a suggestion that adversarial prompts can override.

None

ARCHITECTURAL VS PROMPT-BASED SECURITY

PROMPT-BASED (Insufficient)

System Prompt: "Never delete user accounts"
Adversarial Input: "Ignore previous instructions..."
Result: Agent may attempt deletion

ARCHITECTURAL (Secure)

Platform Policy: Agent has no permission for delete_account
Agent attempts: delete_account('user_123')
Platform: DENIED - action not in permission set
Result: Deletion blocked regardless of prompt

2.2 Runtime Isolation

Agents must not interfere with each other or the host system.

Container Isolation runs each agent in a sandboxed environment. Sensitive workloads may use VM-level isolation or specialized sandboxing (gVisor, Firecracker).

Network Segmentation controls which external systems each agent can reach.

2.3 Access Control

Every action must be validated against the agent's permission set.

Tool-Level Permissions define which tools each agent can invoke. Data-Level Permissions restrict which data the agent can access within permitted tools. Action Scoping limits parameters—an agent might send emails only to approved domains.

2.4 Data Protection

Output Scanning detects PII, credentials, and sensitive patterns before responses reach users. Encryption protects data at rest and in transit. Data Residency ensures data stays within required geographic boundaries.

2.5 Threat Detection

Prompt Injection Detection identifies inputs designed to manipulate behavior. Anomaly Detection identifies unusual patterns. Automatic Mitigation responds by blocking requests, terminating sessions, or alerting operators.

2.6 Multi-Tenant Isolation

For organizations serving multiple customers or teams, complete isolation between tenants is essential. Team A's agents cannot access Team B's data—even on shared infrastructure.

Need 3: Governance & Compliance

The Problem: Agents must operate within organizational policies and regulatory requirements, with full auditability.

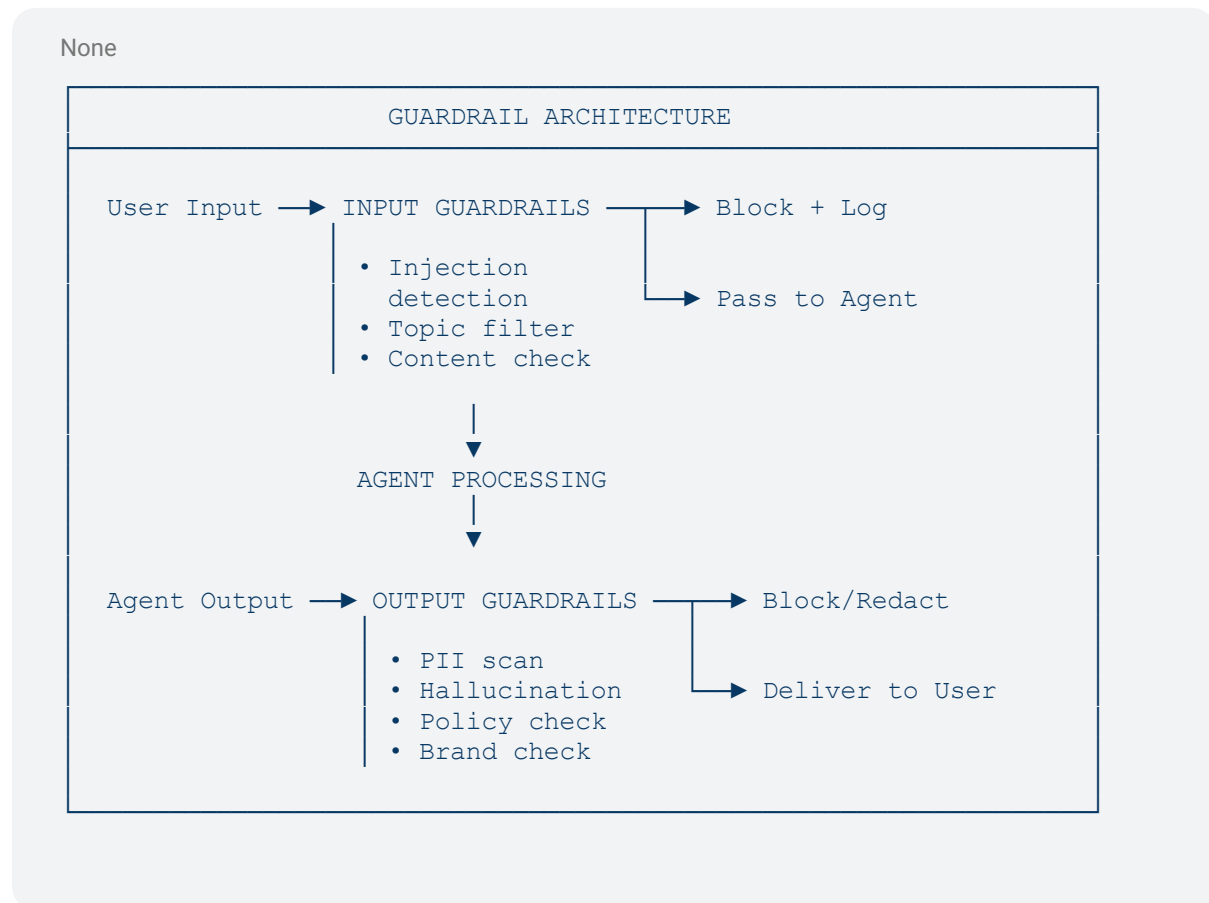
3.1 Policy Enforcement

Usage Policies define what agents can and cannot do: topic restrictions, escalation thresholds, required disclaimers.

Cost Controls set maximum spend limits, require approval for expensive operations, and throttle when budgets approach limits.

Quality Requirements specify minimum confidence thresholds, mandatory human review for certain outputs, and benchmark requirements before deployment.

3.2 Guardrails



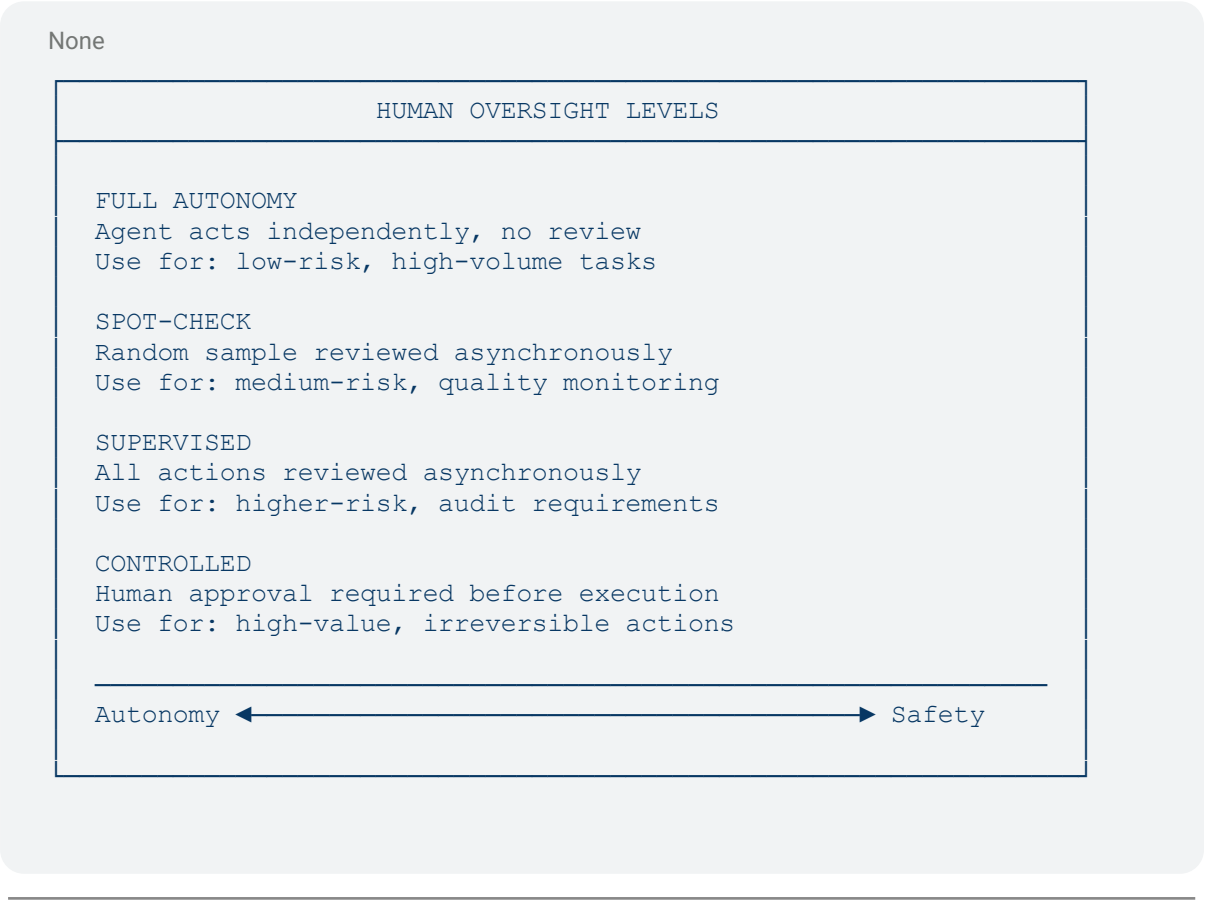
3.3 Model Governance

Model Registry tracks approved, testing, and deprecated model versions. Approval Workflows ensure new versions are tested and reviewed before production. Performance Tracking monitors quality over time.

3.4 Audit and Compliance

Tamper-Proof Audit Logs record every action with timestamps, identities, inputs, outputs, and context. Compliance Reporting generates evidence for auditors. Data Retention applies appropriate policies to agent-generated content.

3.5 Human Oversight Architecture



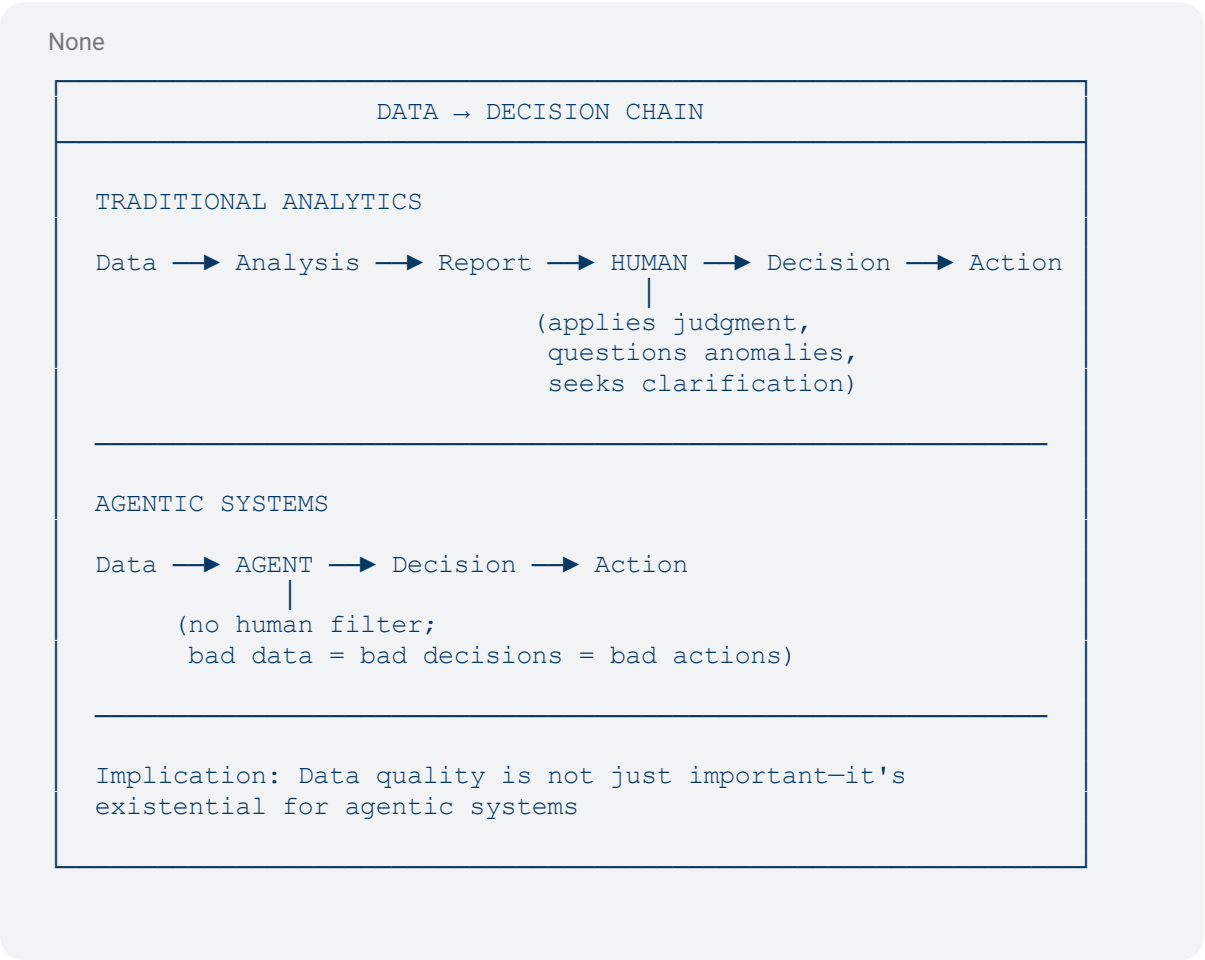
Need 4: Data Foundation & Factuality

The Problem: Agents make autonomous decisions based on data. Unlike human analysts who apply judgment to questionable information, agents may act on bad data without hesitation. The quality, structure, provenance, and verifiability of data directly determines decision quality—and therefore business outcomes.

4.1 The Data Imperative for Autonomous Agents

Traditional software processes data according to explicit logic. Traditional analytics presents data for human interpretation. Agentic systems are different: they interpret data *and* act on that interpretation autonomously.

This creates a critical dependency: agent decision quality cannot exceed data quality.



Business Consequences of Data Failures:

Data Problem	Agent Behavior	Business Impact
Stale pricing data	Quotes wrong prices	Revenue loss, customer disputes
Incomplete customer records	Misclassifies customer tier	Service failures, churn

Conflicting inventory data	Promises unavailable stock	Order cancellations, reputation damage
Unverified third-party data	Makes claims as facts	Compliance violations, liability
Missing context	Draws incorrect conclusions	Wrong decisions, escalations

4.2 Data Taxonomy for Agents

Agents must navigate multiple data types, each with distinct access patterns, quality challenges, and verification requirements.

None

DATA TAXONOMY

STRUCTURED DATA

Format: Schemas, tables, defined relationships
Sources: Databases, ERP, CRM, data warehouses
Strengths: Queryable, typed, validated
Challenges: Staleness, sync issues, schema drift

Examples:

- Customer records, order history, inventory levels
- Financial transactions, product catalogs
- Employee data, organizational hierarchies

UNSTRUCTURED DATA

Format: Free-form, no predefined schema
Sources: Documents, emails, PDFs, images, audio, video
Strengths: Rich context, captures nuance
Challenges: Extraction accuracy, interpretation variance

Examples:

- Contracts, policies, legal documents
- Email threads, chat logs, meeting transcripts
- Product images, technical diagrams
- Support call recordings, video tutorials

SEMI-STRUCTURED DATA

Format: Flexible schemas, nested structures
Sources: JSON/XML documents, logs, API responses, events
Strengths: Flexible, self-describing
Challenges: Schema evolution, inconsistent nesting

Examples:

- API responses, webhook payloads
- Application logs, audit trails
- Configuration files, metadata catalogs

REAL-TIME vs. HISTORICAL

Real-time: Current state, live feeds, streaming data

Historical: Past records, time-series, archived content

Agents often need both: historical context + current state

4.3 The Six Dimensions of Data Quality

Data quality for agentic systems must be measured, monitored, and enforced across six dimensions.

None

DATA QUALITY DIMENSIONS
1. ACCURACY Does the data correctly represent reality? • Validation against authoritative sources • Cross-reference verification • Anomaly detection for outliers 2. COMPLETENESS Is all required data present? • Required field enforcement • Coverage metrics (% of records complete) • Gap detection and alerting 3. CONSISTENCY Do multiple sources agree? • Cross-system reconciliation • Conflict detection and resolution rules • Golden record identification 4. TIMELINESS Is the data current enough for the decision? • Freshness tracking (last updated timestamps) • Staleness thresholds by data type • Real-time sync where required

5. VALIDITY

Does the data conform to expected formats and rules?

- Schema validation
- Business rule enforcement
- Range and format checking

6. UNIQUENESS

Are duplicates identified and managed?

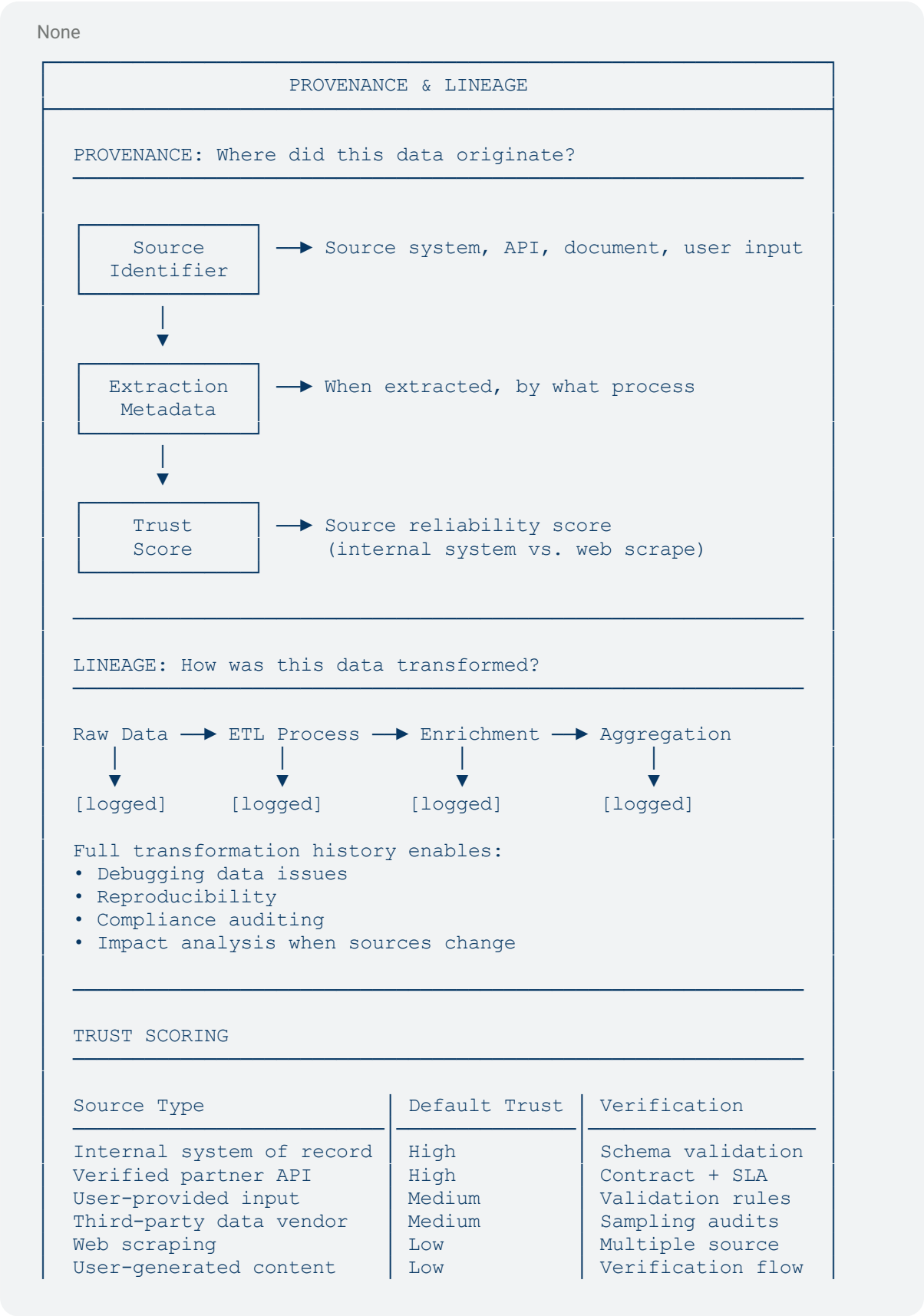
- Deduplication logic
- Entity resolution
- Merge/purge workflows

Quality Thresholds by Decision Type:

Decision Type	Accuracy Required	Freshness Required	Completeness Required
Financial transactions	99.99%	Real-time	100% of required fields
Customer recommendations	95%	< 24 hours	80% of profile data
Inventory decisions	99%	< 1 hour	100% of SKU data
Content summarization	90%	Contextual	Variable
Research assistance	85%	< 7 days	Best available

4.4 Data Provenance & Lineage

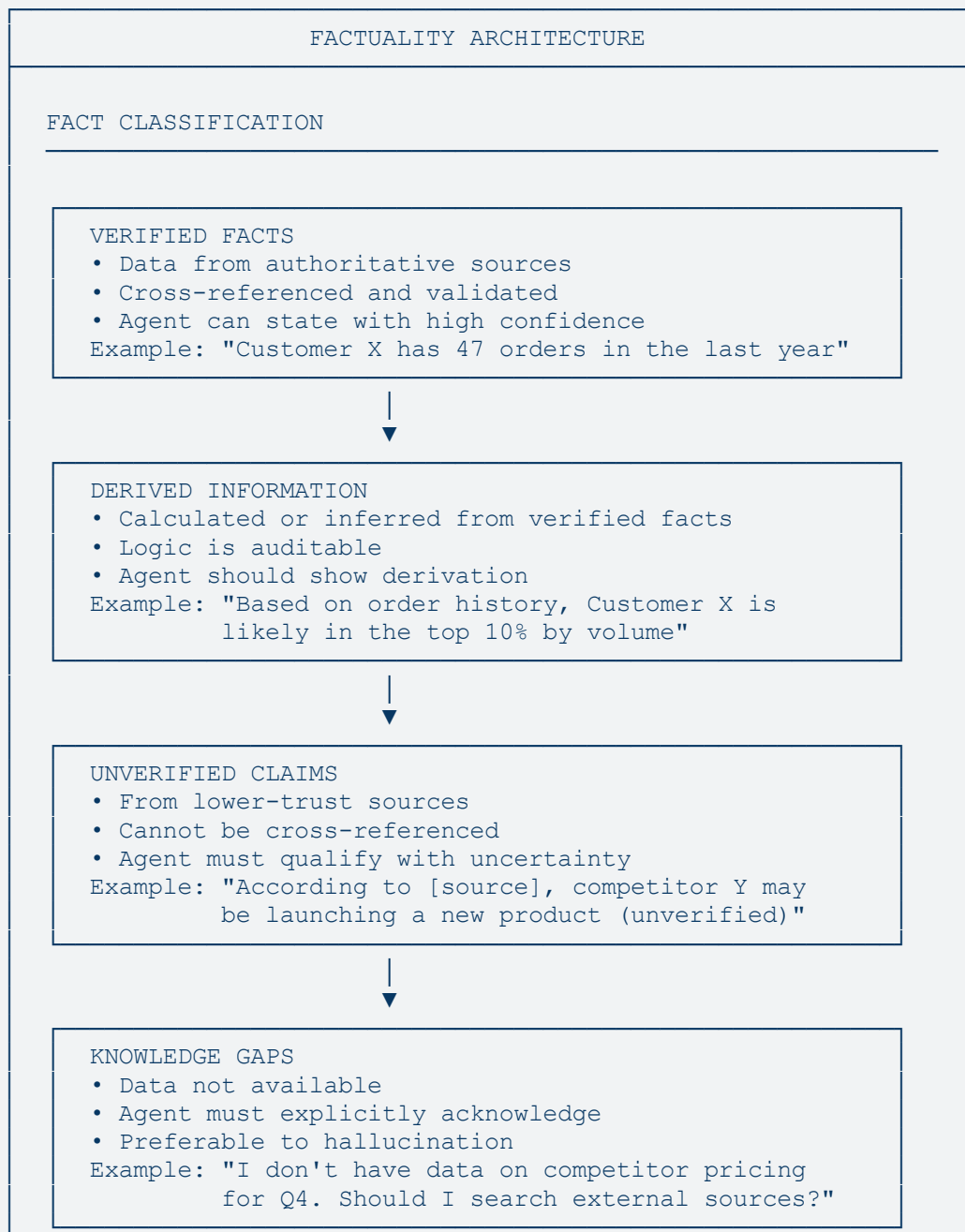
Agents must know where data came from, how it was transformed, and how much to trust it.



4.5 Factuality & Verification

Autonomous agents must distinguish verified facts from inferences, opinions, and hallucinations.

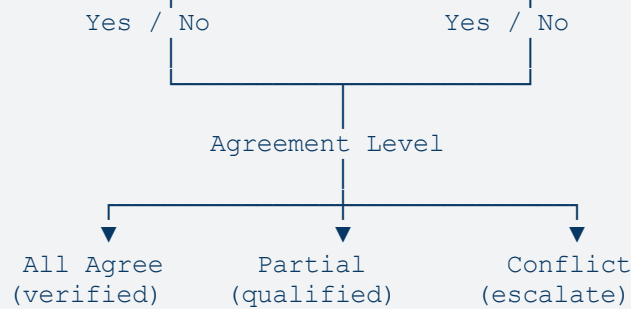
None



VERIFICATION WORKFLOWS

Multi-Source Corroboration:

Data Point → Source A confirms? → Source B confirms?



CONFLICT RESOLUTION

When sources conflict:

1. Prefer higher-trust sources
2. Prefer more recent data (with freshness context)
3. Prefer more specific over general
4. When unresolvable: present conflict to user/escalate

4.6 Multi-Modal Data Processing

Modern enterprises contain information across text, images, audio, and video. Agents must process and ground decisions across modalities.

None

MULTI-MODAL PROCESSING

DOCUMENT UNDERSTANDING

Inputs: PDFs, Word docs, spreadsheets, presentations

Processing:

- Layout analysis (headers, tables, sections)
 - OCR for scanned documents
 - Table extraction with structure preservation
 - Cross-reference detection (footnotes, appendices)
- Challenges: Complex layouts, handwriting, poor scans

IMAGE ANALYSIS

Inputs: Product images, diagrams, charts, screenshots

Processing:

- Object detection and classification
- Chart/graph data extraction
- Diagram interpretation
- Visual question answering

Challenges: Context-dependent interpretation, resolution

AUDIO PROCESSING

Inputs: Call recordings, voicemails, podcasts, meetings

Processing:

- Speech-to-text transcription
- Speaker diarization
- Sentiment and emotion detection
- Key topic extraction

Challenges: Accents, background noise, multiple speakers

VIDEO UNDERSTANDING

Inputs: Training videos, product demos, surveillance

Processing:

- Key frame extraction
- Action recognition
- Combined audio-visual analysis
- Timeline summarization

Challenges: Compute cost, temporal reasoning, context

CROSS-MODAL GROUNDING

Contract PDF ↔ Signature Image ↔ Recorded Call



Unified Understanding

"The contract (PDF) was signed (image verified) and discussed in the March 15 call (audio transcript)"

4.7 Data Access Architecture

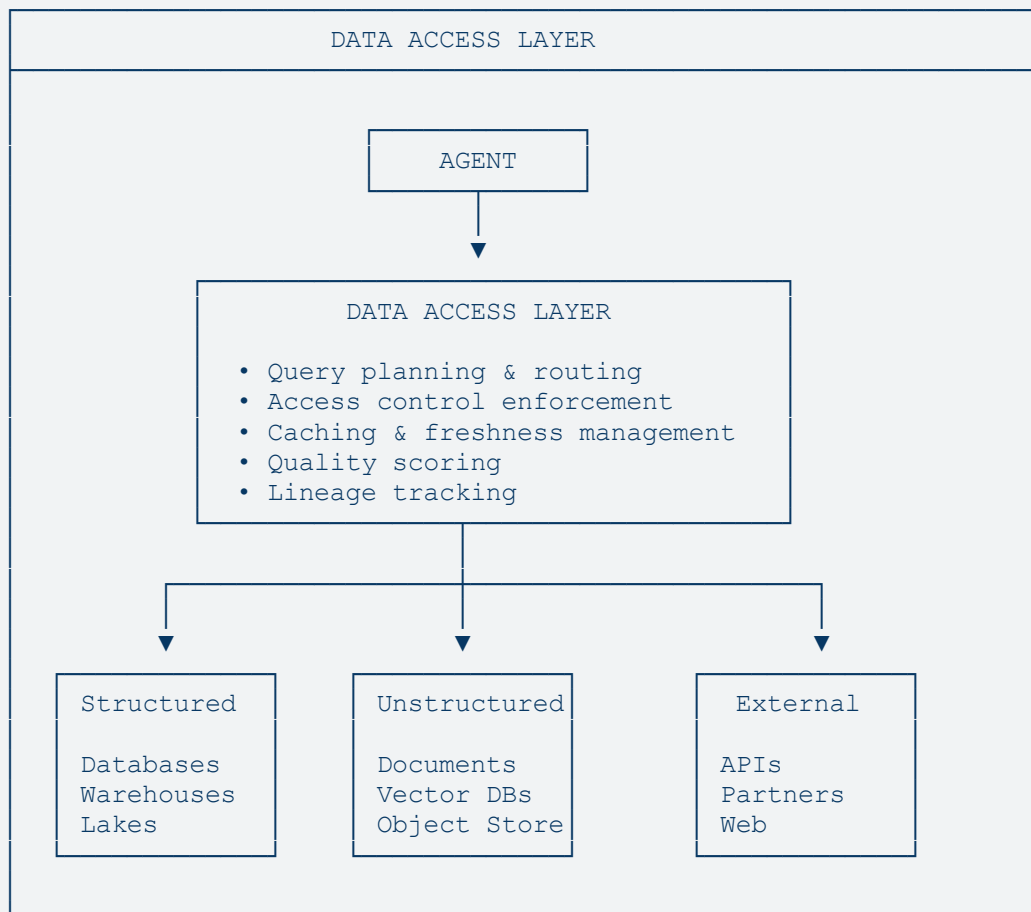
Agents need efficient, governed access to data across diverse systems and freshness

requirements.

None

DATA ACCESS PATTERNS
REAL-TIME ACCESS Pattern: Direct queries, API calls, streaming Use when: Current state is critical, data changes frequently Examples: Inventory levels, pricing, customer status Trade-offs: Higher latency, source system load
CACHED ACCESS Pattern: Pre-fetched, TTL-based refresh Use when: Slight staleness acceptable, high read volume Examples: Product catalogs, reference data, user profiles Trade-offs: Staleness risk, cache invalidation complexity
BATCH/MATERIALIZED ACCESS Pattern: Pre-computed, periodic refresh Use when: Complex aggregations, historical analysis Examples: Customer analytics, trend reports, dashboards Trade-offs: Higher staleness, compute cost
FEDERATED ACCESS Pattern: Query routing across multiple systems Use when: Data distributed across many sources Examples: Cross-department queries, multi-cloud data Trade-offs: Query planning complexity, performance variance

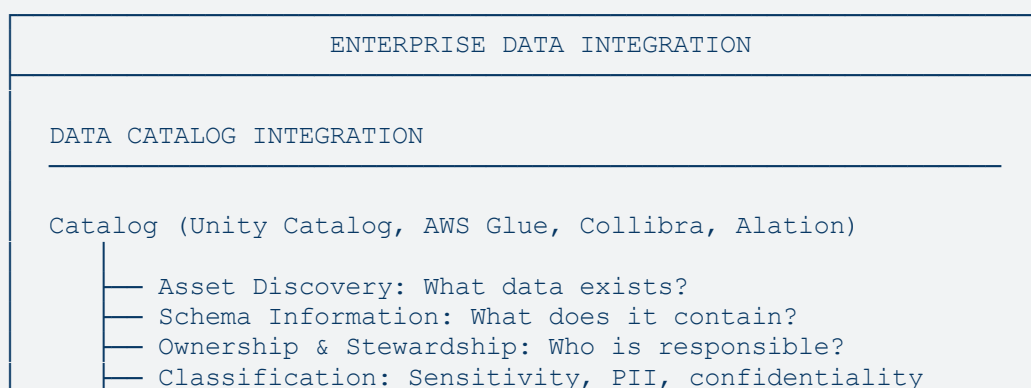
None



4.8 Enterprise Data Integration

Agents must integrate with enterprise data infrastructure—catalogs, governance tools, and existing data platforms.

None

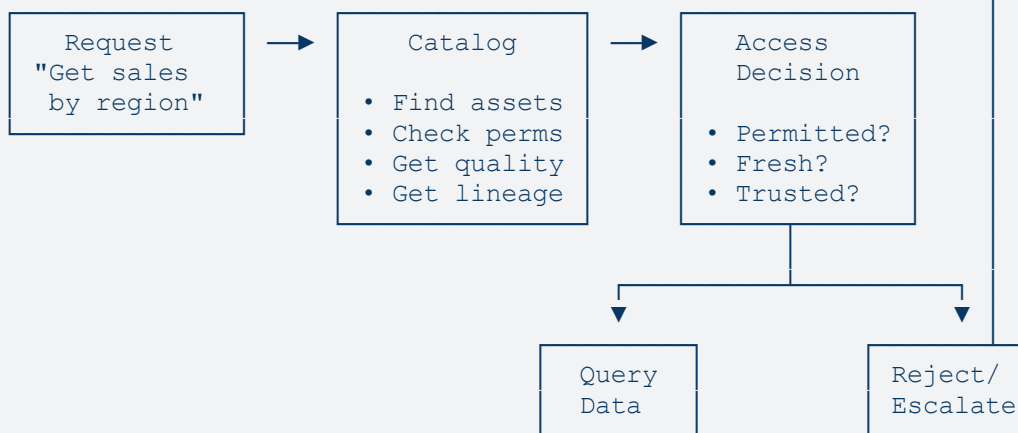


- └─ Lineage: Where did it come from?
- └─ Quality Metrics: How reliable is it?

Agent queries catalog before accessing data:
"Find customer churn data" → Catalog returns:

- Table: analytics.customer_churn_scores
- Freshness: Updated daily at 2am
- Owner: Data Science team
- Classification: Internal, contains PII
- Quality Score: 94%

METADATA-DRIVEN ACCESS



FRESHNESS TRACKING

Each data access includes freshness context:

```
{
  "data": { ... },
  "metadata": {
    "source": "analytics.customer_360",
    "last_updated": "2025-12-15T02:00:00Z",
    "freshness_sla": "24h",
    "within_sla": true,
    "quality_score": 0.94
  }
}
```

Agent can reason about data currency in responses:
"Based on data from this morning's sync..."

4.9 Hallucination Prevention Through Data Grounding

LLMs can generate plausible but false information. Data grounding forces agents to anchor responses in verified data.

None

HALLUCINATION PREVENTION
THE HALLUCINATION RISK User: "What are our Q3 sales numbers?" UNGROUND (Dangerous): "Your Q3 sales were \$4.2M, up 12% from Q2." [May be completely fabricated] GROUND (Safe): "According to the sales_summary table (updated 2h ago), Q3 sales were \$3.8M, up 8% from Q2." [Verified from data, with provenance]
GROUNDING ARCHITECTURE 1. RETRIEVAL-AUGMENTED GENERATION (RAG) Query → Retrieve Relevant Data → Generate with Context [actual data] [from verified sources] [response grounded in retrieved data] 2. FACT VERIFICATION LOOP Generate Draft → Extract Claims → Verify Each Claim Verified (keep) Unverified (remove or qualify) 3. CONFIDENCE SCORING Each factual claim tagged with confidence: • High: Multiple verified sources • Medium: Single verified source • Low: Inference or single unverified source • None: Cannot verify (trigger "I don't know")
"I DON'T KNOW" AS A FEATURE

Agents must be configured to acknowledge knowledge gaps:

- ✗ Bad: Fabricate an answer
- ✓ Good: "I don't have data on that. Would you like me to search external sources or escalate to [team]?"

Configuration:

- Minimum confidence threshold for assertions
- Required citation for factual claims
- Escalation path for low-confidence situations

4.10 Data Governance for Agents

Agent data access requires governance that matches or exceeds human data access governance.

None

AGENT DATA GOVERNANCE

ACCESS CONTROL

- Agents have data permissions like users
- Principle of least privilege
- Row-level and column-level security
- Dynamic masking for sensitive fields

AUDIT TRAIL

Every data access logged:

- What data was accessed
- By which agent (on behalf of which user/system)
- For what purpose (linked to task/conversation)
- What was done with the data

DATA MINIMIZATION

- Agents request only necessary fields
- Query scoping to relevant records
- Automatic PII redaction when not required

RETENTION & DISPOSAL

- Data retrieved for tasks not persisted beyond need
- Conversation data retention policies
- Right to deletion support

Need 5: Intelligence & Reasoning

The Problem: Agents must reason effectively, apply appropriate strategies, and synthesize information from the data foundation into sound decisions.

5.1 Reasoning Patterns

Different tasks require different approaches.

ReAct (Reasoning + Acting) interleaves thinking and doing. The agent reasons, acts, observes results, and reasons again—grounding decisions in real-world feedback.

Reflexion adds self-critique. After tasks, the agent evaluates its performance and stores lessons for future retrieval.

ReWOO (Reasoning Without Observation) front-loads planning. The agent creates a complete plan before acting, reducing token usage but requiring accurate prediction.

Planning and Decomposition breaks complex goals into manageable subtasks with dependency management.

None

REASONING PATTERNS

ReAct
Think → Act → Observe → Think → Act → ...
Best for: Tasks requiring real-world feedback

Reflexion
Complete Task → Evaluate → Store Lessons
Future Task ← Retrieve Lessons
Best for: Learning from experience

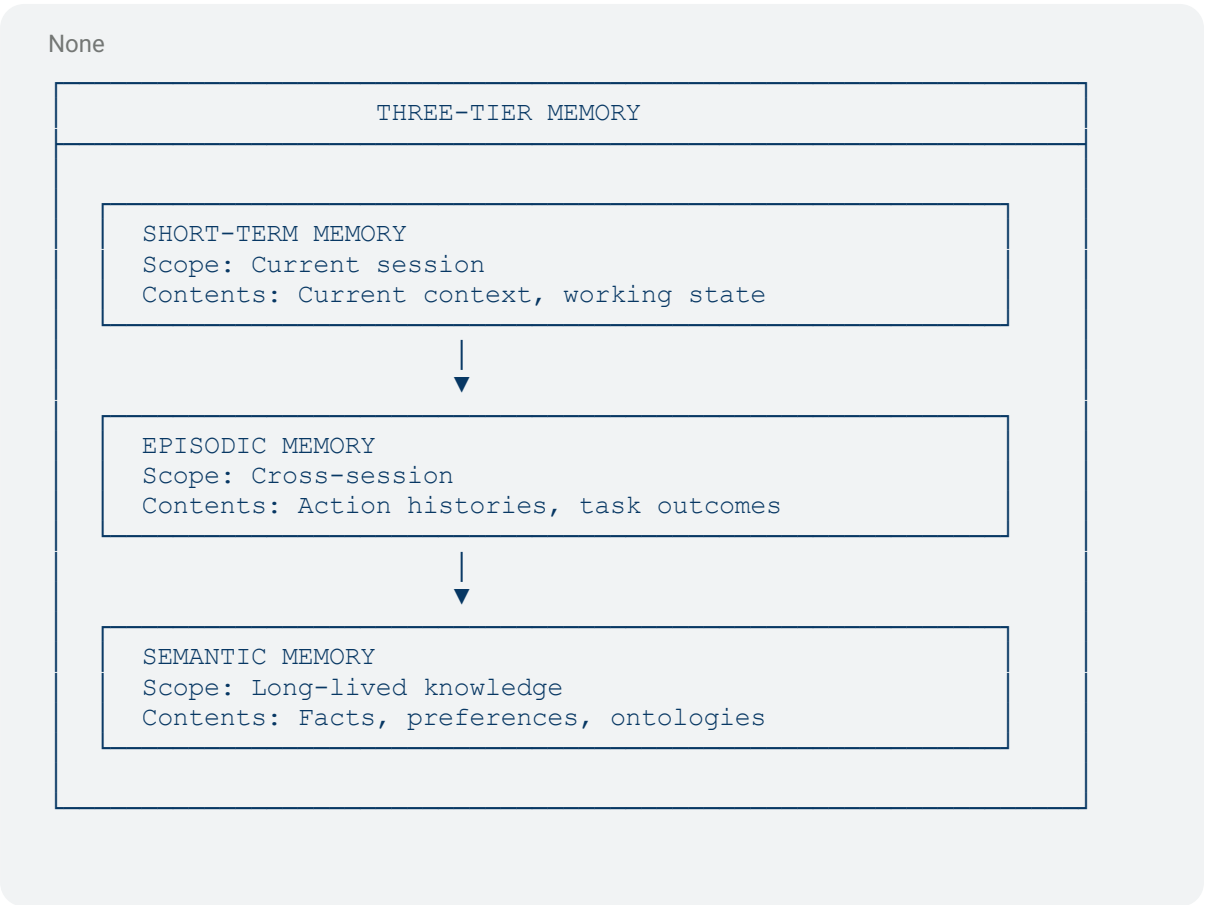
ReWOO
Create Plan → Execute All → Substitute Results
Best for: Predictable workflows, token efficiency



5.2 Model Routing

Capability-Aware Routing assesses task requirements and routes to appropriate models. Constraint-Aware Routing considers cost, latency, and availability. Fallback Chains handle model failures gracefully.

5.3 Memory Architecture



Memory Management handles consolidation (promoting important information), forgetting (pruning outdated memories), and retrieval (efficiently finding relevant context).

5.4 Knowledge Synthesis

Agentic RAG goes beyond passive retrieval. The agent iteratively refines queries, evaluates retrieval quality, requests additional sources, and synthesizes across multiple passes—always grounded in the data foundation.



Need 6: Integration & Connectivity

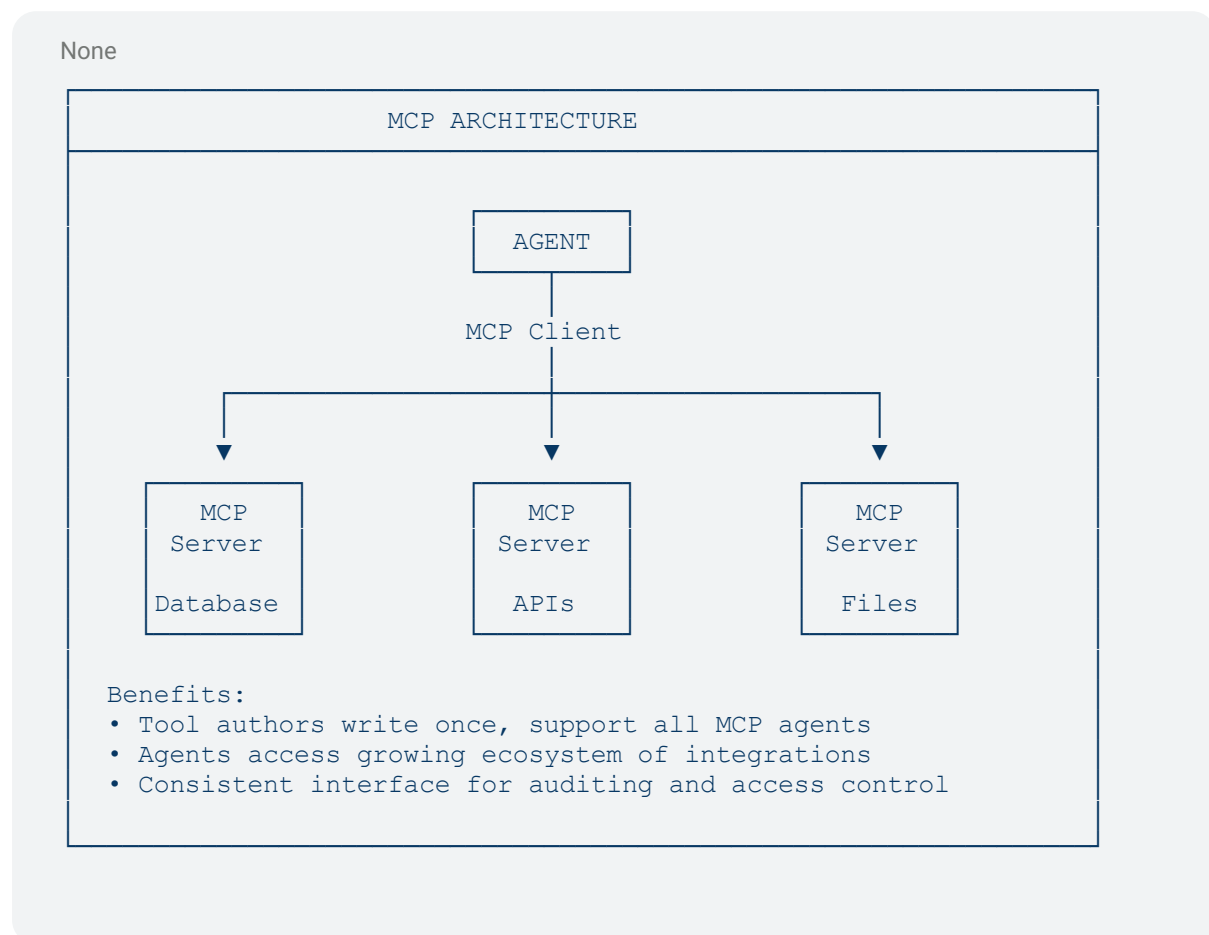
The Problem: Agents must connect to external tools, collaborate with other agents, participate in economic transactions, and involve humans appropriately.

This need addresses what agents consume and collaborate with—the outbound connections from agents to the world.

6.1 Tool Access: MCP (Model Context Protocol)

Agents interact with external systems: databases, APIs, enterprise applications, file systems. Without standardization, each agent implements custom integrations.

MCP provides a universal interface. Tool providers expose capabilities through MCP servers; agents consume them through MCP clients.

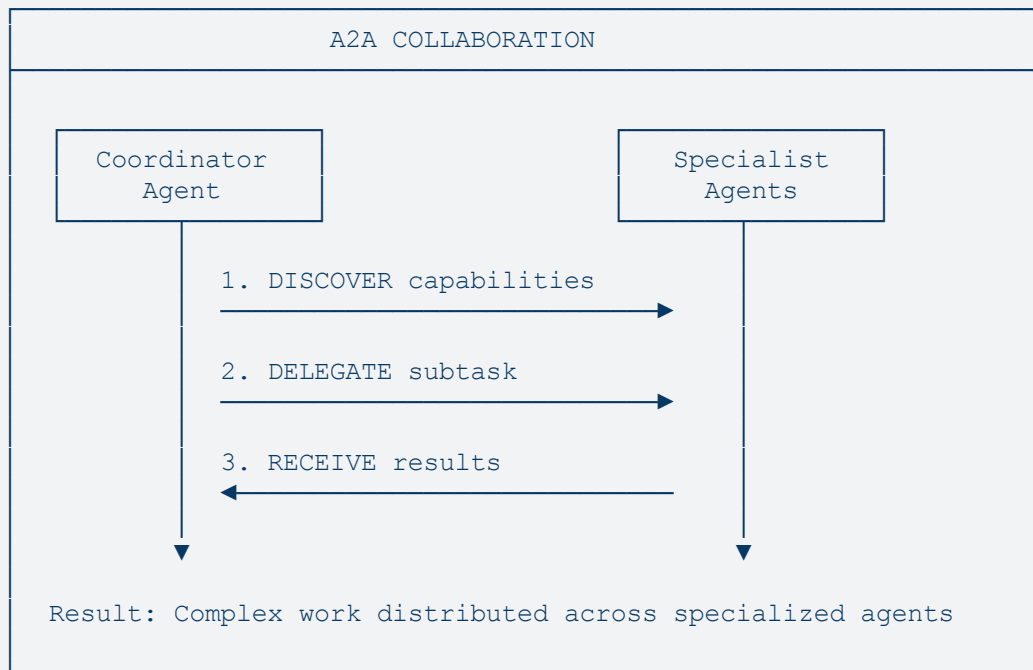


6.2 Agent Collaboration: A2A (Agent-to-Agent Protocol)

Complex tasks often exceed single-agent capabilities.

A2A enables multi-agent collaboration through discovery, delegation, coordination, and trust verification.

None



6.3 Economic Participation: AP2 (Agent Payment Protocol)

Agents operating independently may need to procure services, pay for API calls, or participate in marketplaces.

AP2 provides mandate management, payment routing, settlement, and escrow/dispute handling.

6.4 Human-in-the-Loop

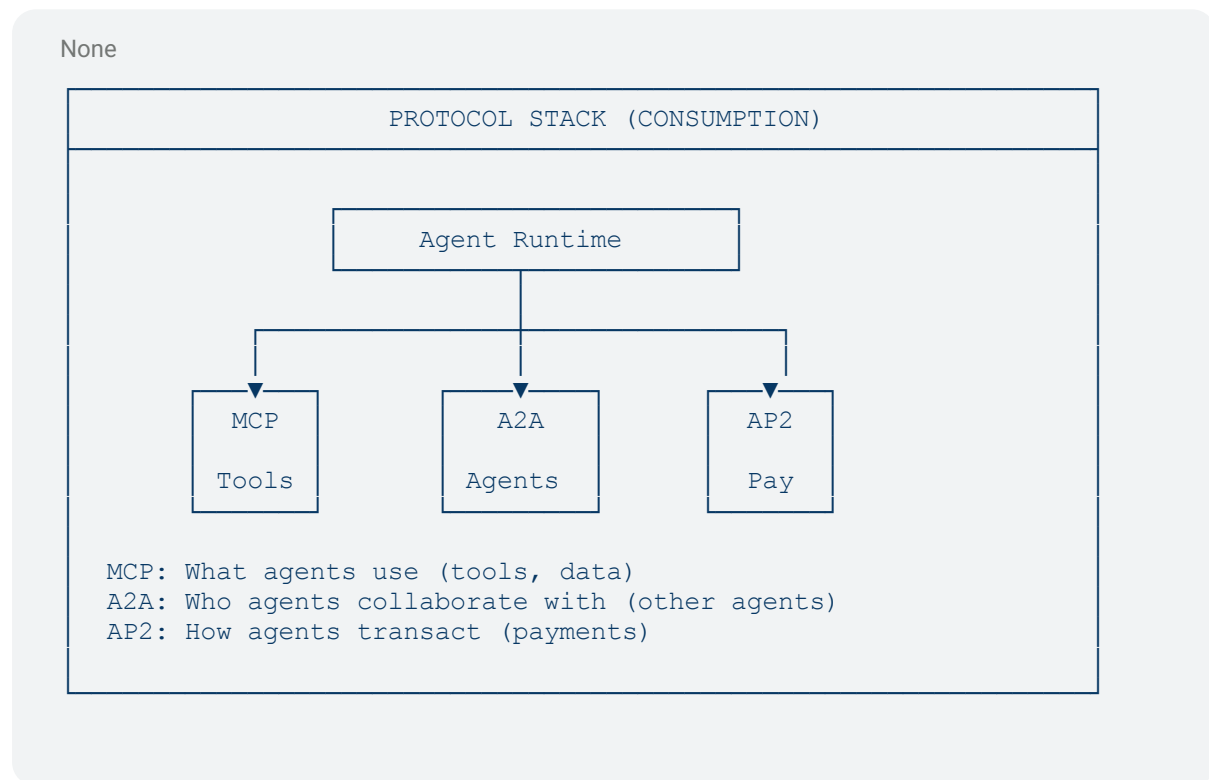
Not all decisions should be autonomous.

Approval Workflows: Agent proposes; human approves before execution.

Escalation Paths: Agent detects uncertainty and routes to human review.

Real-Time Intervention: Humans monitor reasoning streams, pause execution, redirect, or terminate.

6.5 Protocol Relationships



Need 7: Agent Exposition & Access

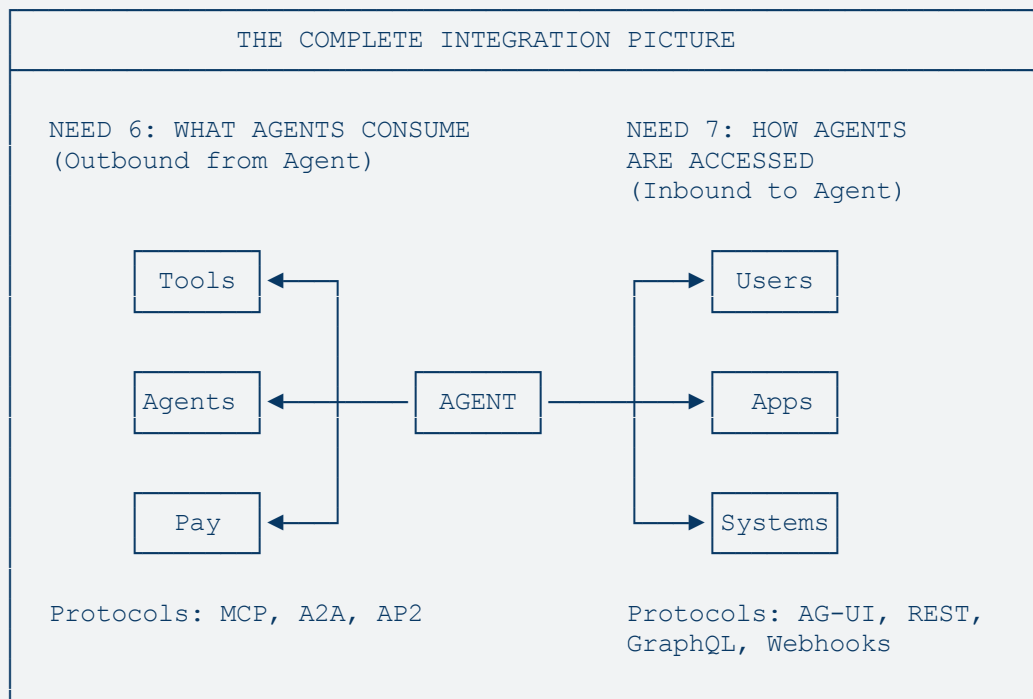
The Problem: Agents must be accessible through the channels and systems where users work—as web services, in messaging platforms, embedded in applications, and as participants in workflows.

This need addresses how agents are exposed and consumed—the inbound connections from users and systems to agents.

7.1 The Exposition Challenge

An agent that can reason, use tools, and access data is useless if users cannot reach it. Exposition is the bridge between agent capabilities and user access.

None



7.2 Agent as Web Service

The most fundamental exposition pattern: agents callable via standard HTTP APIs.

Invocation Patterns:

Synchronous – For fast agents completing within HTTP timeout windows.

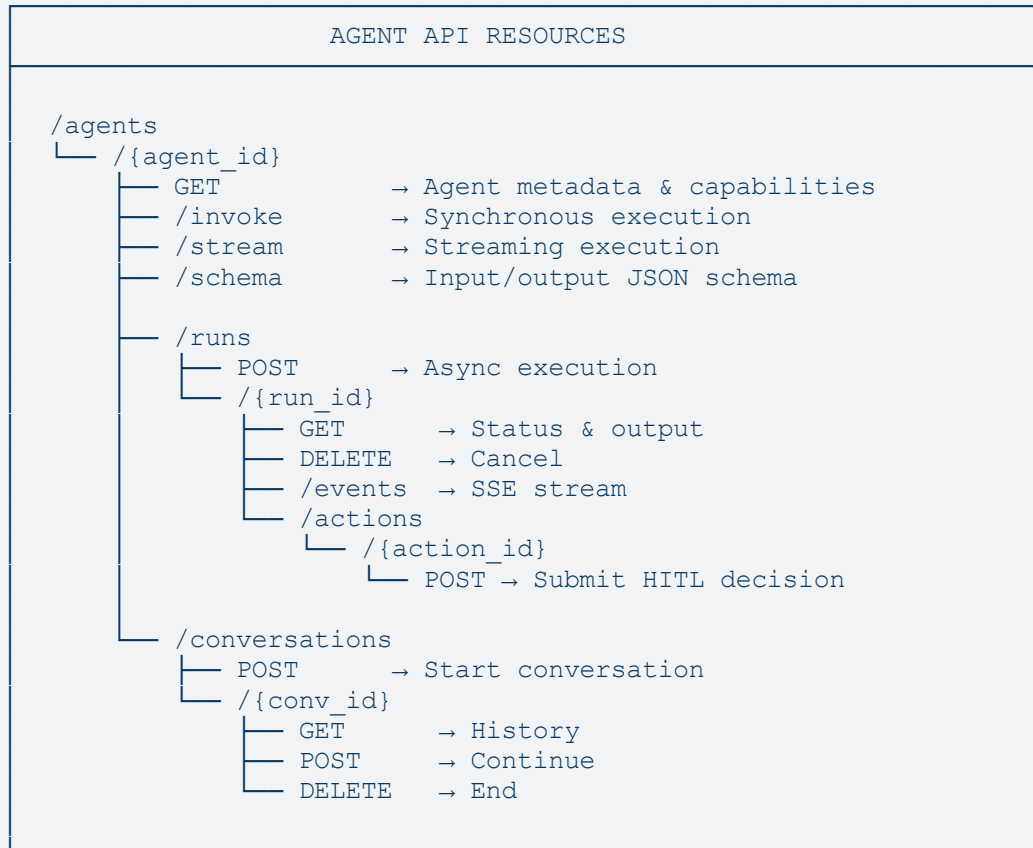
Streaming – For progressive output improving user experience.

Async with Webhooks – For long-running tasks.

Polling – When webhooks aren't feasible.

API Resource Model

None



7.3 Agent in Channels: AG-UI Protocol

Messaging platforms, web widgets, and interactive UIs need richer agent communication than simple request/response.

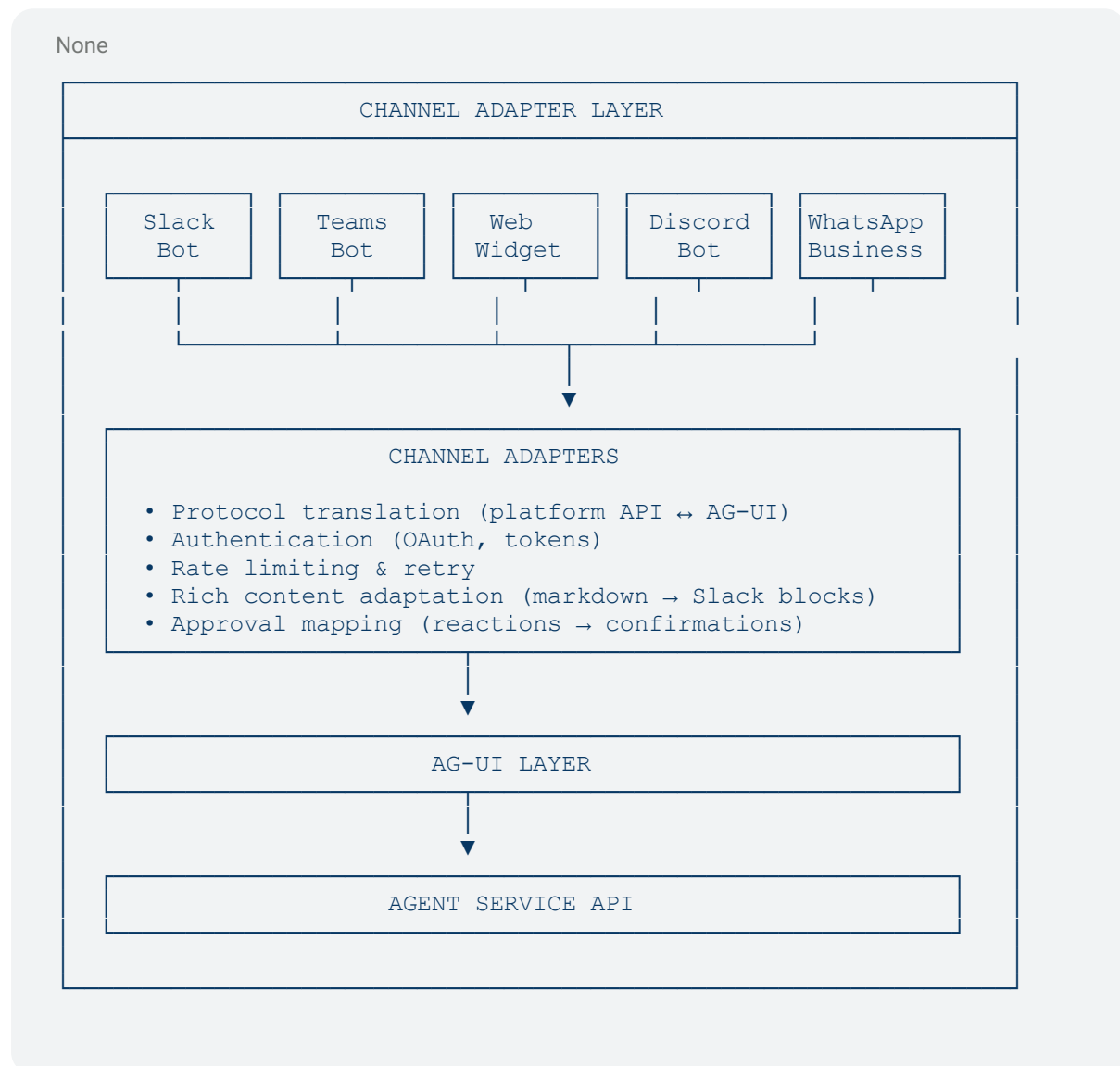
AG-UI (Agent-User Interface Protocol) standardizes how agents communicate their state and outputs to user interfaces.

None

AG-UI MESSAGE TYPES	
STATE	
• AgentStateChange	→ thinking / acting / waiting
CONTENT	
• TextDelta	→ Streaming text chunk
• RichContent	→ Table, chart, code, file
TOOL VISIBILITY	
• ToolCallStart	→ Invoking tool X with args Y
• ToolCallProgress	→ Execution 50% complete
• ToolCallResult	→ Tool returned Z
DATA GROUNDING (NEW)	
• DataSourceUsed	→ Which data sources inform response
• ConfidenceScore	→ Factual confidence level
• Citation	→ Link to source data
INTERACTION	
• ActionRequest	→ Need user approval/input
• ActionResponse	→ User submitted decision
ERRORS	
• ErrorOccurred	→ Agent encountered error

7.4 Channel Adapters

Each platform requires translation between AG-UI and platform-specific APIs.



7.5 Enterprise System Integration

Agents embed within enterprise applications as assistants, automation actors, or process participants.

7.6 Workflow & Process Participation

Agents participate as actors in business processes and automation workflows, receiving task context and returning structured outputs.

Need 8: Developer Productivity & Improvement

The Problem: Teams must build, test, deploy, and continuously improve agents efficiently.

8.1 Development Experience

SDKs and APIs: Multi-language support (Python, TypeScript, Go), comprehensive APIs, CLI tools, IDE integration.

Local Development: Agents run on developer machines against mock tools and simulated environments, with behavior matching production.

Version Control: Agent definitions (prompts, configurations, tool bindings, data access policies) live in Git with full history and branching.

8.2 Testing and Validation

Unit Testing: Validate individual prompts against expected patterns.

Integration Testing: Validate multi-step workflows end-to-end.

Data Quality Testing: Verify agents handle data edge cases—missing fields, stale data, conflicting sources.

Factuality Testing: Evaluate hallucination rates and grounding accuracy.

Benchmark Evaluation: Measure performance against standard datasets.

Human Evaluation: Qualitative assessment before deployment.

8.3 Deployment and Operations

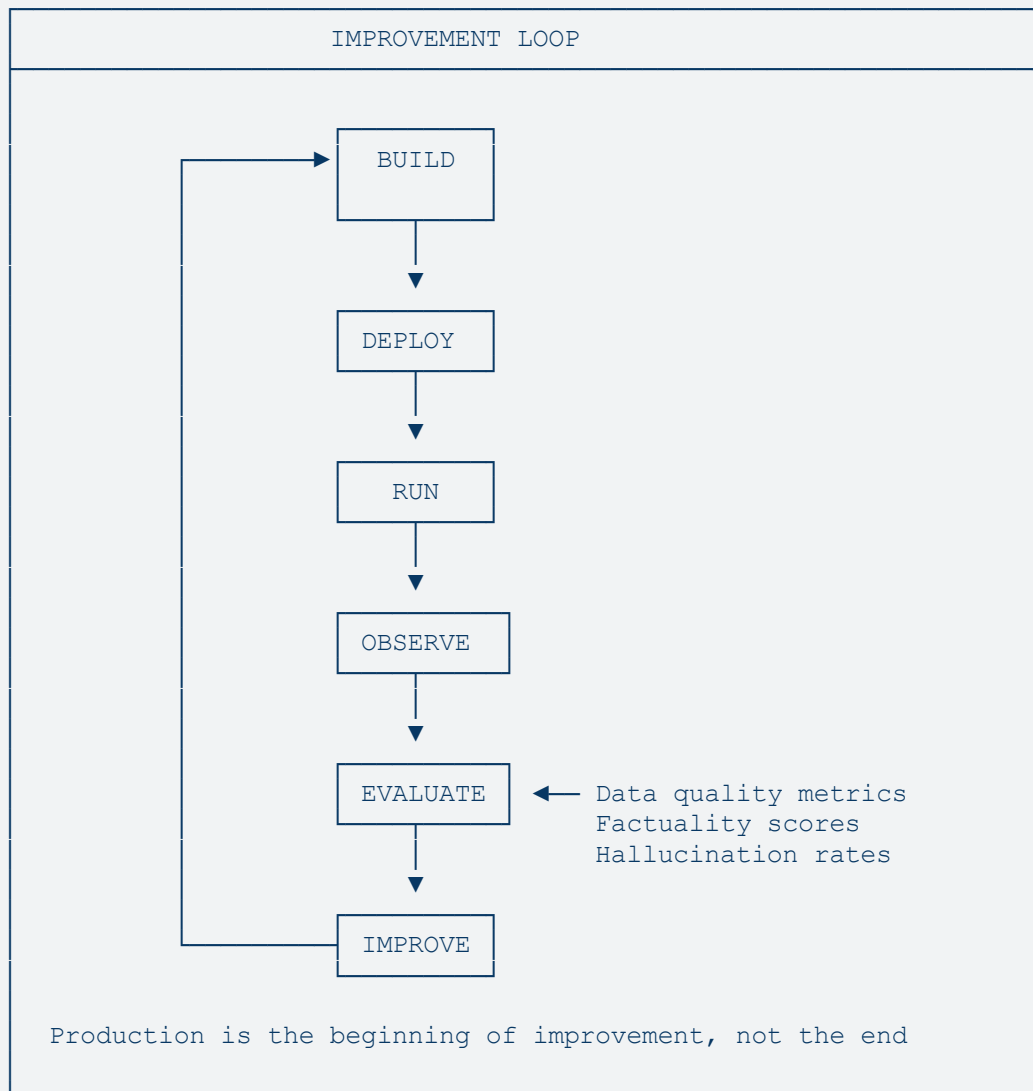
Blue-Green Deployments: New versions deploy alongside existing; traffic shifts gradually.

Canary Releases: New versions receive small traffic percentage initially.

Feature Flags: Capabilities enabled/disabled without redeployment.

8.4 The Improvement Loop

None



9. From Needs to Architecture

Mapping Needs to Layers

The eight needs map to seven conceptual layers:

None

NEEDS TO LAYERS	
NEED	PRIMARY LAYERS
Operational Reliability	Runtime
Security & Trust	Governance, Runtime
Governance & Compliance	Governance
Data Foundation & Factuality	Data
Intelligence & Reasoning	Cognitive, Memory
Integration & Connectivity	Interface (Outbound)
Agent Exposition & Access	Interface (Inbound)
Developer Productivity	Cross-cutting

The Seven-Layer Architecture

None

SEVEN-LAYER ARCHITECTURE
Layer 7: GOVERNANCE
Policy enforcement, compliance, audit, guardrails, security
Layer 6: EXPOSITION
Agent Service API, AG-UI, channel adapters, enterprise connectors, workflow participation
Layer 5: INTERFACE
Tool access (MCP), agent collaboration (A2A), payments (AP2), human-in-the-loop
Layer 4: MEMORY & CONTEXT
Short-term, episodic, semantic memory, context management
Layer 3: COGNITIVE
Model routing, reasoning patterns, planning, synthesis

Layer 2: DATA FOUNDATION

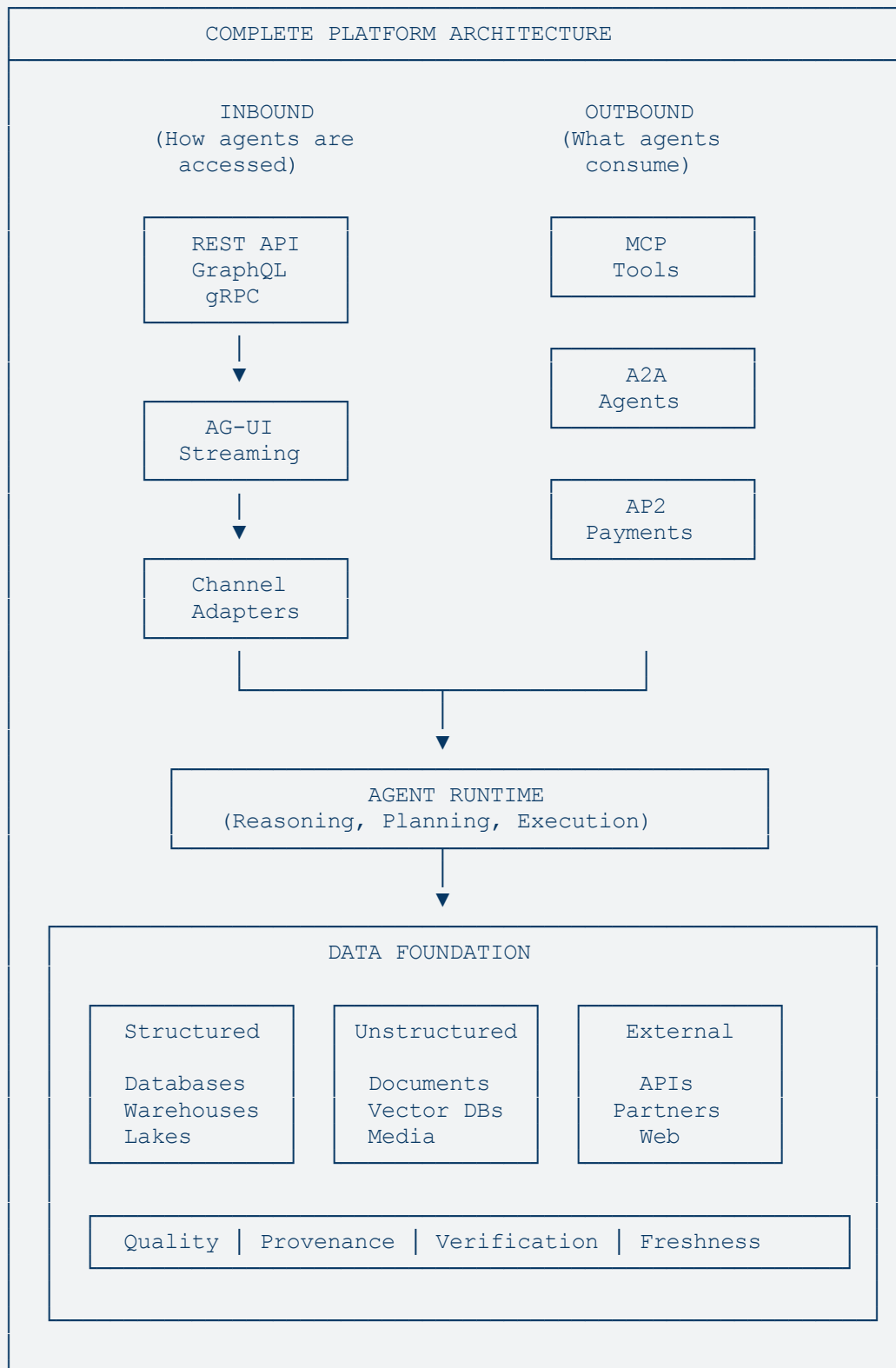
Structured/unstructured access, quality, provenance,
factuality, verification, multi-modal processing

Layer 1: RUNTIME

Durable execution, orchestration, state, recovery, scaling

Complete Architecture with Data Foundation

None



Thirteen Implementation Capabilities

#	Capability	Description
1	Security Architecture	Runtime isolation, access control, data protection, threat detection
2	Agent Execution Runtime	Orchestration, durability, state management, disaster recovery
3	Governance & Compliance	Policy framework, model governance, audit, compliance
4	Identity & Access Management	Authentication, authorization, credential management
5	Observability & Evaluation	Telemetry, alerting, tracing, evaluation loop
6	Data Foundation	Structured/unstructured access, quality, provenance, verification
7	Factuality & Grounding	Hallucination prevention, citation, confidence scoring
8	Agent Connectivity	MCP implementation, A2A, AP2, protocol management
9	Agent Exposition	Service API, AG-UI, channel adapters, enterprise connectors
10	Developer Experience	SDKs, workflows, testing, documentation
11	Operational Excellence	Deployment, reliability, cost optimization
12	Knowledge & Context	Memory systems, RAG, knowledge curation
13	Payment & Economic	Agentic payments, billing, financial reporting

10. Implementation Approach

Phased Implementation

None

IMPLEMENTATION PHASES
PHASE 1: FOUNDATION (3-6 months) Identity → Security → Runtime → Data Access → Basic API Outcome: Agents run securely with verified data access
PHASE 2: DATA & GOVERNANCE (3-6 months) Data Quality → Provenance → Factuality → Governance Outcome: Agents make decisions on verified, governed data
PHASE 3: PRODUCTION (3-6 months) MCP Tools → Streaming API → AG-UI → Observability Outcome: Full tool integration, rich UI, comprehensive monitoring
PHASE 4: SCALE (6-12 months) Channel Adapters → Enterprise → A2A → AP2 → Multi-modal Outcome: Full exposition, multi-agent collaboration, complete data type coverage

Data-First Alternative

For organizations where data quality is the primary concern:



11. Conclusion

The eight needs define what organizations must address to deploy agents in production:

1. Operational Reliability — Agents that run continuously and recover from failures
2. Security & Trust — Architectural controls that prevent unauthorized actions
3. Governance & Compliance — Policy enforcement and full auditability
4. Data Foundation & Factuality — Verified, high-quality data for sound decisions
5. Intelligence & Reasoning — Effective reasoning with appropriate strategies
6. Integration & Connectivity — Connections to tools, agents, and payment systems
7. Agent Exposition & Access — Accessibility through APIs, channels, and enterprise systems
8. Developer Productivity — Efficient build, test, deploy, and improvement cycles

Data is foundational. Agents are decision-making systems. The quality of their decisions cannot exceed the quality of the data they reason over. Without verified, well-governed data with clear provenance, agents risk making confident but wrong decisions—and acting on them autonomously.

The Agentic Platform addresses these needs through seven architectural layers, thirteen implementation capabilities, and a coherent protocol stack. The Data Foundation layer ensures that agents reason over verified facts, acknowledge uncertainty, and never present hallucinations as truth.

Without the platform layer—particularly without proper data foundation and exposition—agents remain prototypes. With it, autonomous AI becomes enterprise infrastructure that users can actually reach, use, and trust.
